

DATA WAREHOUSE

Comprehensive Study Notes — Foundations to Advanced Topics

1. Fundamentals of Data Warehousing

1.1 Definition & Purpose

A Data Warehouse (DWH) is a subject-oriented, integrated, non-volatile, and time-variant collection of data in support of management's decision-making process (Inmon, 1990). It is the backbone of enterprise Business Intelligence (BI).

Key Purpose: Enable analytical querying over large, historical, integrated datasets
Primary Consumers: Business analysts, data scientists, executives, BI tools
NOT Designed For: High-frequency inserts/updates, transactional operations (OLTP)

1.2 Inmon's Four Pillars

Property	Definition	Implication
Subject-Oriented	Data organized around core business subjects — Sales, Finance, HR, Products	Avoids process-centric silos; unifies data by entity
Integrated	Data from multiple sources cleansed and standardized into one consistent format	Resolves naming conflicts, encoding differences, unit mismatches
Non-Volatile	Once loaded, data is never deleted or updated in-place	Enables reproducible reports; historical records remain intact
Time-Variant	Every fact carries a time dimension; snapshots are preserved over time	Enables trend analysis, year-over-year comparisons, audits

1.3 OLTP vs. OLAP

Dimension	OLTP	OLAP
Full Name	Online Transaction Processing	Online Analytical Processing
Purpose	Execute business operations	Support analytical decisions
Query Type	Simple INSERT / UPDATE / DELETE / SELECT	Complex aggregations, GROUP BY, joins, window functions

Dimension	OLTP	OLAP
Data Volume per Query	A few rows	Millions to billions of rows
Response Time	< 100 ms	Seconds to minutes
Schema Design	Highly normalized (3NF/BCNF)	Denormalized (Star, Snowflake)
Data Currency	Real-time / near-real-time	Historical (hours to months old)
Concurrency	Thousands of transactions/sec	Tens to hundreds of analytical queries
Backup Strategy	Frequent, point-in-time recovery	Periodic; can rebuild from source
Example Systems	MySQL, PostgreSQL, Oracle	Snowflake, BigQuery, Redshift

2. Data Warehouse Architectures

2.1 Inmon Architecture (Top-Down / CIF)

Developed by Bill Inmon. The Corporate Information Factory (CIF) model builds a single, normalized enterprise data warehouse first, then derives data marts from it.

Flow: Source Systems → ETL → Enterprise DW (normalized, 3NF) → Data Marts → BI Tools
✓ Advantages: Single version of truth, easier governance, less data redundancy
✗ Disadvantages: High upfront cost, long time-to-value, requires heavyweight planning
Best For: Large enterprises with strict governance and compliance requirements

2.2 Kimball Architecture (Bottom-Up / Bus Architecture)

Developed by Ralph Kimball. Build dimensional data marts first, integrate them via conformed dimensions (the 'bus'), and together they form the enterprise warehouse.

Flow: Source Systems → ETL → Dimensional Data Marts (Star Schemas) → BI Tools
✓ Advantages: Fast time-to-value, user-friendly, iterative delivery
✗ Disadvantages: Risk of inconsistency without discipline; harder to enforce enterprise standards
Best For: Agile environments, departmental BI, organizations with limited DW maturity

2.3 Data Vault Architecture

Developed by Dan Linstedt (2000). A hybrid methodology combining Inmon normalization with Kimball flexibility. Built for auditability, scalability, and parallel loading.

Component	Purpose	Contains
Hub	Stores unique business keys	Business key, load date, record source
Link	Captures relationships between hubs	Foreign keys to Hubs, load date, record source
Satellite	Stores descriptive attributes & history	Hub/Link FK, load date, end date, attributes

✓ Strengths: Full audit trail, parallel loading, schema flexibility, handles late-arriving data
✗ Weaknesses: More complex SQL, many tables, requires tooling to be practical
Best For: Regulated industries (Banking, Insurance), audit-heavy environments

2.4 Lambda & Kappa Architectures (Streaming + Batch)

Lambda Architecture

- Batch Layer: Processes all historical data; produces accurate but delayed views
- Speed Layer: Processes real-time streams; provides low-latency approximate results
- Serving Layer: Merges batch and speed views for query serving

Challenge: Maintaining two codebases (batch + streaming) for the same logic.

Kappa Architecture

Simplification of Lambda — treats everything as a stream. Historical reprocessing is done by replaying the event log (e.g., Kafka topic). Single codebase for all processing.

2.5 Federated / Virtual Data Warehouse

No physical consolidation — a virtual layer (e.g., Presto, Trino, Dremio) queries data across multiple disparate sources in real-time, presenting a unified view. Useful when data cannot be moved due to governance, latency, or compliance.

3. ETL, ELT & Data Pipelines

3.1 ETL — Extract, Transform, Load

The traditional approach. Data is extracted from sources, transformed in an intermediate processing engine, then loaded into the warehouse. Dominant in on-premise architectures.

Phase	Activities	Tools
Extract	Pull data from RDBMS, flat files, APIs, mainframes, CDC streams	Informatica PowerCenter, Talend, SSIS, Apache NiFi, Sqoop
Transform	Cleanse, deduplicate, validate, enrich, conform, aggregate, apply business rules	Spark, SSIS, dbt (staging), Informatica transformations
Load	Bulk insert or incremental upsert into target DW tables	COPY command, BCP, sqlldr, bulk loader APIs

3.2 ELT — Extract, Load, Transform

Modern cloud-first approach. Raw data is loaded into the warehouse first (cheap cloud storage), then transformed using the warehouse's compute power (SQL). dbt is the dominant transformation tool.

Key Insight: Cloud warehouses have massive compute — transformation inside them is often faster and cheaper than maintaining a separate ETL server
Schema-on-read: Raw data preserved; transformations are replayable and versionable
dbt (data build tool): Manages SQL transformations as version-controlled models with lineage, tests, and documentation

3.3 CDC — Change Data Capture

CDC captures only the rows that changed in source systems, reducing load on source databases and enabling near-real-time data pipelines.

CDC Method	How It Works	Pros / Cons
Log-Based CDC	Reads database transaction logs (binlog, WAL, redo log)	Minimal source impact; complex setup; requires log access
Trigger-Based CDC	DB triggers write changes to a shadow/audit table	Simple; high source DB overhead
Timestamp-Based	Queries rows WHERE updated_at > last_run	Easy; misses deletes; requires reliable updated_at column
Diff-Based	Compares full source snapshot to previous snapshot	No source changes required; very slow for large tables

3.4 Data Pipeline Orchestration

Orchestration tools schedule, monitor, and manage dependencies between pipeline tasks.

Tool	Type	Key Features
Apache Airflow	Open-source	DAG-based, Python-defined, huge ecosystem, Kubernetes executor
Prefect	Modern open-source	Dynamic DAGs, native Kubernetes, better error handling than Airflow

Tool	Type	Key Features
Dagster	Asset-oriented OSS	Software-defined assets, lineage-first, strong observability
dbt Cloud	ELT-specific	Scheduling, CI/CD, docs, lineage for dbt projects
AWS Glue	Managed ETL	Serverless Spark-based ETL on AWS, auto-schema discovery
Azure Data Factory	Managed ETL	Visual pipeline builder, deep Azure integration, 90+ connectors

4. Dimensional Modeling (Kimball)

4.1 Core Concepts

Fact Table	Contains quantitative, measurable events (metrics/KPIs). Each row = one observation at a defined grain. FK references to dimension tables.
Dimension Table	Descriptive context for facts — answers Who, What, Where, When, Why, How. Usually wide and denormalized.
Grain	The finest level of detail a fact table row represents. MUST be declared before designing the table. E.g., 'one row per order line item per day'.

4.2 Fact Table Types

Type	Description	Grain Example	Measures
Transaction Fact	One row per discrete business event	One row per sales line item	Revenue, quantity, discount
Periodic Snapshot	State captured at regular intervals	One row per account per month	Account balance, loan balance
Accumulating Snapshot	Tracks a process through lifecycle stages	One row per order (updated)	Days-to-ship, fulfillment date
Factless Fact	Records events with no numeric measure	Student course enrollment	Existence only (count)
Aggregate Fact	Pre-summarized for performance	Monthly sales by region	Sum of revenue, avg order value

4.3 Schema Types

Star Schema

Central fact table surrounded by denormalized dimension tables. Optimal for query performance — minimal joins required.

[Date Dim] — [FACT: Sales] — [Customer Dim]
[Product Dim]
[Store Dim]
✓ Fewer joins, faster queries, simpler SQL, easier for BI tools
✗ Slight data redundancy in dimension tables

Snowflake Schema

Dimension tables are normalized into sub-dimensions (e.g., Product → Category → Department). Reduces redundancy but increases join complexity.

✓ Less storage, reduced redundancy, easier dimension maintenance
✗ More joins = slower queries, harder for analysts to understand
Use when storage is extremely constrained or dimension hierarchies are deep and complex

4.4 Slowly Changing Dimensions (SCD)

SCDs handle how descriptive dimension attributes change over time. Choosing the right SCD type is a critical design decision.

SCD Type	Strategy	History?	Use Case
Type 0 — Fixed	Never update the attribute	No	Attributes that should never change (e.g., birth date)
Type 1 — Overwrite	Replace old value with new value	No	Corrections, unimportant history (e.g., fix typo in name)
Type 2 — Version Row	Add new row; flag old row as expired	Full	Most common: track full history of customer address, product price
Type 3 — New Column	Add column for previous value	Partial (1 prior)	Only last change matters (e.g., previous department)
Type 4 — History Table	Separate mini-dimension for history	Full	Very frequently changing attributes (e.g., status codes)
Type 6 — Hybrid (1+2+3)	New row + overwrite current + add prev column	Full + current flag	Best of all worlds; complex to implement

SCD Type 2 Implementation Detail

- Add surrogate key as PK (integer, auto-increment)
- Add effective_start_date and effective_end_date columns
- Add is_current flag (1 = active, 0 = expired)
- When attribute changes: set end_date + is_current=0 on old row; insert new row with new attribute and is_current=1

4.5 Advanced Dimension Concepts

Concept	Definition	Example
Conformed Dimension	Shared dimension used across multiple fact tables/marts with identical meaning	Date dimension shared by Sales, HR, and Finance facts
Role-Playing Dimension	Same physical dimension referenced multiple times in one fact table under different aliases	Date dim used as Order Date, Ship Date, Delivery Date
Junk Dimension	Single table holding miscellaneous low-cardinality flags/indicators to reduce fact table width	Transaction type, payment flag, promo flag
Degenerate Dimension	Dimension attribute stored directly in the fact table (no separate dim table)	Order number, invoice number
Outrigger Dimension	A secondary dimension attached to a primary dimension (not the fact)	Zip Code details hanging off Customer dimension
Bridge Table	Junction table enabling many-to-many relationships in dimensional models	Customer-Account, Student-Course
Mini-Dimension	Separates rapidly changing attributes from a large, slow-changing dimension	Customer demographics (age band, income band) spun out from Customer dim

5. Advanced Design Patterns

5.1 Bus Matrix (Kimball)

A planning tool mapping fact tables (rows) against shared dimensions (columns). An 'X' indicates the fact table uses that dimension. Ensures conformed dimensions are identified upfront.

Fact Table	Date	Customer	Product	Store	Employee
Sales Transactions	X	X	X	X	
Inventory Snapshot	X		X	X	
HR Payroll	X				X
Web Sessions	X	X			

5.2 Late Arriving Data

Late Arriving Facts

A fact arrives after the event date. The system must look up the correct dimension surrogate key that was active at event_time, not current time.

- Use effective date in dimension lookup: WHERE event_date BETWEEN eff_start AND eff_end AND is_current = 0
- Reprocess affected aggregate tables if pre-computed

Late Arriving Dimensions

A dimension row arrives after the fact that references it (its natural key exists in the fact but not in the dimension).

- Insert an 'inferred member' placeholder row with unknown attributes
- When real dimension row arrives, back-fill attributes and link to the same surrogate key

5.3 Data Vault 2.0 Advanced Concepts

DV 2.0 Concept	Description
Point-in-Time (PIT) Tables	Snapshot of current satellite row pointers at a given time — speeds up time-travel queries across multiple satellites
Bridge Tables	Pre-joins across hub-link-hub paths for query performance
Reference Tables	Static lookup data (country codes, currencies) managed separately from main vault
Same-As Link	Resolves duplicate business keys from different source systems
Computed Satellite	Derived/aggregated data stored in satellite form for performance

5.4 Anchor Modeling

An ultra-flexible, 6NF-like approach where every attribute gets its own table. Provides extreme flexibility for schema evolution at the cost of very high join complexity. Rarely used in practice but important for highly volatile schemas.

6. Storage Formats & Performance Optimization

6.1 Columnar vs. Row Storage

Aspect	Row Storage	Columnar Storage
Storage Layout	Full rows stored together	Each column stored separately
Best For	OLTP (fetch entire row)	OLAP (aggregate few columns over many rows)
Compression	Lower (mixed data types per block)	High (same data type per block — RLE, dictionary)

Aspect	Row Storage	Columnar Storage
Query Performance	Slow for analytics (reads unused columns)	Very fast for analytics (reads only needed columns)
Examples	PostgreSQL, MySQL heap files	Snowflake, BigQuery, Redshift, Parquet, ORC

6.2 File Formats for Data Warehousing

Format	Type	Compression	Schema	Best For
Parquet	Columnar, binary	Snappy/ZSTD/GZIP	Embedded	Analytical workloads, Spark, Hive, BigQuery
ORC	Columnar, binary	ZLIB/Snappy	Embedded	Hive, Presto; better for predicate pushdown
Avro	Row, binary	Snappy/Deflate	Separate (JSON)	Kafka serialization, schema evolution
Delta Lake	Columnar (Parquet)+TX log	Snappy/ZSTD	Embedded + catalog	ACID on data lakes; time travel
Apache Iceberg	Table format (any file)	Any	Catalog-based	Multi-engine lake tables, time travel, schema evolution
Apache Hudi	Columnar + index	Any	Catalog-based	Upserts on data lakes, CDC ingestion
CSV/JSON	Row, text	None (gzip optional)	None (inferred)	Ingestion only; avoid for analytics

6.3 Partitioning Strategies

Strategy	Description	Best For
Range Partitioning	Rows divided by value ranges (e.g., date ranges by month/year)	Time-series data; most common in DW
List Partitioning	Rows divided by discrete values (e.g., country, region)	Known finite set of categorical values
Hash Partitioning	Rows assigned by hash of key; even distribution	Uniform distribution; avoid skew for large dimension joins
Composite	Combination of two strategies (e.g., Range + Hash)	Very large tables needing both pruning and distribution
Dynamic Partition Pruning	Query planner eliminates partitions not matching WHERE clause	Critical for performance; all modern cloud DWs support this

6.4 Indexing in Analytical Systems

Index Type	Description	Cardinality	Use Case
Bitmap Index	Bit vector per value; AND/OR operations are bitwise	Low (< 100 distinct values)	Gender, status flags in OLAP (Oracle DW, Vertica)
B-Tree Index	Balanced tree; excellent for equality and range lookups	High	Primary/unique keys, point lookups
Clustered Index	Data physically ordered by index key	Any	Range scans on sorted column (SQL Server, MySQL InnoDB)
Zone Map / Min-Max	Stores min/max per storage block; skips blocks outside range	N/A	Columnar DWs (Snowflake micro-partitions, Redshift zone maps)
Bloom Filter	Probabilistic structure; answers 'does this value exist?'	High	Skip irrelevant files in lake formats (Parquet, Iceberg)

6.5 Materialized Views & Aggregates

Pre-computed query results stored physically. Dramatically speed up repeated or predictable analytical queries.

- Full Refresh MV: Entire view rebuilt on refresh schedule — simple but expensive
- Incremental MV: Only delta (new/changed) rows appended — efficient but complex
- Automatic Query Rewrite: Some DWs (BigQuery, Oracle) transparently route queries to MVs without SQL changes
- Aggregate Navigation: BI layer automatically selects the most granular pre-aggregate that satisfies the query

6.6 Query Optimization Techniques

Technique	Description
Predicate Pushdown	Filter conditions pushed as close to the data source as possible, reducing I/O
Projection Pushdown	Only needed columns read from columnar files (column pruning)
Join Reordering	Optimizer reorders joins to start with smallest filtered result set
Broadcast Join	Small table replicated to all nodes; eliminates shuffle for large-small joins
Sort-Merge Join	Both tables sorted on join key then merged; good for large-large joins
Result Caching	Identical queries served from cache (Snowflake caches 24 hrs by default)
Adaptive Query Execution	Runtime re-optimization based on actual statistics mid-query (Spark 3.x, BigQuery)
Approximate Aggregation	HyperLogLog, Count-Min Sketch, T-Digest for near-instant approximate answers on massive data

7. Cloud Data Warehouses — Deep Dive

7.1 Comparison of Major Platforms

Feature	Snowflake	BigQuery	Redshift	Synapse Analytics
Provider	Snowflake Inc. (multi-cloud)	Google Cloud	AWS	Microsoft Azure
Architecture	Separated storage + compute	Serverless, fully managed	MPP cluster (or Serverless)	MPP + Spark (unified)
Scaling	Virtual Warehouses, auto-scale	Automatic, per-query	Manual node add/remove or serverless	Dedicated or serverless pool
Storage Format	Proprietary columnar (FDN)	Capacitor (columnar)	Proprietary columnar	Parquet on ADLS
Pricing Model	Credit-based compute + storage	Per-query (TB scanned) + storage	Node-hour or serverless	DWU-hour or serverless
Unique Feature	Time Travel, Zero-Copy Clone, Data Sharing	ML in SQL (BQML), external tables	AQUA accelerator, RA3 nodes	Deep Azure integration, Spark
Concurrency	Multi-cluster warehouses	Auto-managed	WLM queues	Workload groups

7.2 Snowflake Architecture Deep Dive

Three-Layer Architecture

- Cloud Services Layer: Query compilation, optimization, metadata, access control, transactions
- Query Processing Layer: Virtual Warehouses (VWs) — ephemeral MPP clusters of compute nodes
- Database Storage Layer: Data stored in S3/GCS/Azure Blob as compressed, columnar micro-partitions

Key Snowflake Concepts

Concept	Description
Micro-Partitions	Immutable, compressed columnar storage units (50–500 MB compressed). Zone maps stored per micro-partition for pruning.
Virtual Warehouse (VW)	An MPP cluster of compute nodes. T-shirt sized (XS to 6XL). Scales independently of storage. Can suspend/resume in seconds.

Concept	Description
Multi-Cluster Warehouse	Multiple VW clusters behind one endpoint; auto-scales to handle concurrency spikes.
Time Travel	Query data as it existed at any point in the past (default 1 day, up to 90 days on Enterprise). Uses immutable micro-partition history.
Zero-Copy Clone	Instant clone of table/schema/database. No data is copied — clone references same micro-partitions until writes diverge.
Data Sharing	Share live data with external Snowflake accounts without copying or ETL. Consumer reads from provider's storage.
Search Optimization Service	Maintains a secondary index structure for fast point lookups on large tables. Billed separately.
Dynamic Data Masking	Column-level security policy that masks sensitive data at query time based on user role.

7.3 BigQuery Architecture Deep Dive

- Dremel query engine: Converts SQL to a distributed execution tree across thousands of slots
- Colossus storage: Google's distributed file system storing Capacitor columnar files
- Jupiter network: Ultra-high bandwidth network connecting compute and storage at wire speed
- Slot-based compute: 1 slot $\approx$ 1 vCPU thread. On-demand or committed reservations.
- BQML: Train ML models (linear regression, k-means, XGBoost, TensorFlow) using SQL
- External Tables: Query data in GCS, Bigtable, Google Sheets without loading

8. Data Lake, Lakehouse & Open Table Formats

8.1 Data Lake vs. Data Warehouse vs. Lakehouse

Dimension	Data Lake	Data Warehouse	Data Lakehouse
Data Types	Structured, semi-structured, unstructured	Primarily structured	All types
Schema	Schema-on-read	Schema-on-write	Schema-on-read + enforcement
ACID Transactions	No (raw files)	Yes	Yes (via Delta/Iceberg/Hudi)
Query Performance	Low (raw) / Medium (with engines)	High	High
Cost (Storage)	Very low (object storage)	Higher	Very low
Best Users	Data scientists, ML engineers	Business analysts, BI tools	All personas unified
Technology	S3 + Hive, Presto	Snowflake, Redshift, BigQuery	Delta Lake, Iceberg, Databricks

8.2 Open Table Formats

Delta Lake (Linux Foundation / Databricks)

- Adds a transaction log (`_delta_log`) on top of Parquet files
- Provides ACID transactions, schema enforcement, time travel (RESTORE command)
- OPTIMIZE command compacts small files; VACUUM removes old files
- Native to Databricks; supported by Spark, Presto, Trino, Athena

Apache Iceberg (Netflix / Apple)

- Table format spec — engine-agnostic (Spark, Trino, Flink, Hive all read/write Iceberg)
- Metadata: catalog → metadata.json → manifest list → manifest files → data files (Parquet/ORC/Avro)
- Features: hidden partitioning, partition evolution, schema evolution, time travel, row-level deletes
- Copy-on-Write vs. Merge-on-Read modes for update/delete handling

Apache Hudi (Uber)

- Optimized for upserts and incremental processing from CDC pipelines
- Copy-on-Write (CoW): Rewrites entire Parquet file on each update — good for reads, slow writes
- Merge-on-Read (MoR): Writes delta log files, merges on read — fast writes, slower reads
- Supports Hive, Spark, Presto, Trino; native on AWS EMR

9. Data Governance, Quality & Security

9.1 Data Quality Dimensions

Dimension	Definition	Detection Method
Completeness	No missing required values	NULL counts, fill rate checks
Accuracy	Correctly represents real-world values	Cross-system validation, business rule checks
Consistency	Same values across systems and time	Cross-table reconciliation, referential integrity
Timeliness	Data available when needed by consumers	SLA monitoring, load lag alerts
Uniqueness	No duplicate records	Deduplication counts, PK violation checks
Validity	Conforms to defined formats and domains	Regex checks, range validation, FK checks
Integrity	Relationships between data are valid	FK checks, orphan record detection

9.2 Data Lineage

Data lineage tracks the flow of data from its origin through all transformations to its final consumption point. Critical for impact analysis, debugging, and compliance.

- Column-level lineage: Tracks which source column feeds each target column (gold standard)
- Table-level lineage: Which tables feed which other tables
- Tools: Apache Atlas, OpenLineage, dbt (built-in lineage graph), Alation, DataHub, Marquez

9.3 Master Data Management (MDM)

MDM establishes a single, authoritative 'golden record' for key business entities (Customer, Product, Employee, Supplier) across all systems.

MDM Style	Description	Use Case
Consolidation	Aggregate from all sources; master is read-only to source systems	Reporting master
Registry	Master stores only cross-reference index; no data duplication	Identity resolution
Centralized	Master is the system of record; all systems read from it	Product catalog
Coexistence	Master and source systems coexist and sync bidirectionally	Large enterprises with legacy constraints

9.4 Data Security in the Warehouse

Security Layer	Mechanism	Example
Authentication	SSO, MFA, service accounts, OAuth 2.0	Okta + Snowflake SSO
Authorization	RBAC (Role-Based Access Control), ABAC	Snowflake roles: SYSADMIN, USERADMIN, custom roles
Column-Level Security	Dynamic data masking, column masking policies	Mask SSN to 'XXX-XX-1234' for non-privileged roles
Row-Level Security	Row Access Policies restrict rows visible per role	Sales reps see only their region's data
Data Encryption	Encryption at rest (AES-256) + in transit (TLS 1.2+)	All major cloud DWs enforce this by default
Network Security	VPC, Private Link, IP allowlisting	Snowflake Private Link bypasses public internet
Audit Logging	Query history, access logs, copy activity logs	Snowflake QUERY_HISTORY, BigQuery Data Access Logs

10. The Modern Data Stack (MDS)

10.1 MDS Architecture

INGESTION	STORAGE	TRANSFORM	SEMANTIC	CONSUME
_____	_____	_____	_____	_____
Fivetran /	Snowflake /	dbt Core /	Looker LookML /	Tableau /
Airbyte /	BigQuery /	dbt Cloud	Cube.dev /	Power BI /
Stitch	Redshift	MetriQL	Metabase	
Orchestration: Airflow / Prefect / Dagster				
Observability: Monte Carlo / Soda / Great Expectations				
Catalog: Alation / Atlan / DataHub / dbt Docs				

10.2 dbt (data build tool) — Deep Dive

dbt transforms data inside the warehouse using SQL SELECT statements. It handles dependency management, testing, documentation, and lineage automatically.

dbt Concept	Description
Model	A SQL SELECT statement saved as a .sql file. dbt compiles and executes it as a table or view.
Materialization	How a model is persisted: view (default), table, incremental, ephemeral, or materialized view.
Incremental Model	Only processes new/changed rows. Uses is_incremental() macro + unique_key for upsert logic.
Source	Declares raw source tables with freshness SLAs and schema documentation.
Ref()	Macro that builds the DAG by creating explicit dependencies between models.
Macro	Jinja2 template for reusable SQL logic (e.g., generate_surrogate_key, pivot).
Test	Schema tests (not_null, unique, accepted_values, relationships) + custom SQL tests run after each build.
Seeds	Small CSV files loaded into the warehouse as tables via dbt seed.
Snapshot	SCD Type 2 implementation: dbt tracks row changes using strategy: timestamp or check.
Exposure	Documents downstream consumers (dashboards, ML models) in the lineage graph.
Metric	Defined business metrics (revenue, MAU) queryable via the Metrics Layer / semantic layer.

10.3 Semantic Layer

A semantic layer (also called metrics layer or headless BI) sits between the warehouse and BI tools. It centralizes metric definitions so that 'revenue' always means the same thing regardless of which tool queries it.

- Tools: Cube.dev, dbt Semantic Layer (MetricFlow), Looker LookML, AtScale
- Prevents metric proliferation — one source of truth for KPI definitions
- Enables self-service analytics with governed, consistent data

11. Real-Time Data Warehousing & Streaming

11.1 Micro-Batch vs. Streaming

Approach	Latency	Tools	When to Use
Batch ETL	Hours to daily	Airflow, SSIS, Glue	Non-time-sensitive reporting
Micro-Batch	Minutes	Spark Structured Streaming, AWS Glue Streaming	Near-real-time dashboards
True Streaming	Sub-second	Apache Flink, Kafka Streams, Spark Structured Streaming	Fraud detection, real-time monitoring

11.2 Apache Kafka in the DW Pipeline

Kafka is the dominant event streaming platform for feeding real-time data into warehouses and lakes.

- Producer → Kafka Topic (partitioned, replicated log) → Consumer Groups
- Kafka Connect: Pre-built connectors (JDBC source, S3 sink, Snowflake sink) for no-code integration
- Kafka Streams / ksqlDB: Stream processing directly on Kafka topics before warehouse ingestion
- Schema Registry: Enforces Avro/Protobuf/JSON schema compatibility across producers and consumers

11.3 Streaming Directly into Cloud DWs

Platform	Streaming Mechanism	Detail
Snowflake	Snowpipe + Kafka Connector	Micro-batch; files land in S3/GCS → Snowpipe triggers COPY; ~1 min latency
Snowflake	Snowpipe Streaming API	Row-level streaming SDK; sub-minute latency; uses dynamic tables for transform
BigQuery	Storage Write API	High-throughput streaming with exactly-once semantics
Redshift	Streaming Ingestion from Kinesis	Native Kinesis Data Streams → Redshift materialized view

Platform	Streaming Mechanism	Detail
Databricks	Auto Loader	File discovery on S3/ADLS with schema inference; triggers incremental Spark jobs

12. Metadata Management, Data Catalog & Observability

12.1 Types of Metadata

Metadata Type	What It Covers	Examples
Technical Metadata	Physical structure of data assets	Table names, column types, indexes, row counts, storage size
Business Metadata	Business meaning and context	KPI definitions, data owner, PII classification, SLA
Operational Metadata	Pipeline and process information	ETL run time, rows loaded, last refresh, job status
Process / Lineage Metadata	Data flow and transformation history	Column-level lineage, transformation SQL, upstream sources
Social / Behavioral Metadata	Human interaction with data	Most queried tables, certified datasets, user ratings

12.2 Data Observability

Data observability monitors data health continuously — detecting anomalies in freshness, volume, schema, and distribution before downstream consumers notice.

Pillar	What to Monitor	Alert Example
Freshness	How recently was data updated?	orders table not loaded in 3 hours (SLA breach)
Volume	Row count and data size trends	Daily row count dropped 80% vs. 7-day average
Schema	Column additions, removals, type changes	New NULL column detected in fact_sales
Distribution	Statistical properties of values	Revenue column has 500% spike in max value
Lineage	Impact of upstream changes on downstream	Source table renamed; 12 downstream models affected

13. Data Mesh & Decentralized Architecture

13.1 What is Data Mesh?

Coined by Zhamak Dehghani (2019). A sociotechnical approach that decentralizes data ownership to domain teams, treating data as a product.

13.2 Four Principles of Data Mesh

Principle	Description
1. Domain Ownership	Each business domain (Orders, Payments, Marketing) owns and serves its own data
2. Data as a Product	Domain data is treated as a product — discoverable, addressable, trustworthy, self-describing, interoperable, secure
3. Self-Serve Data Platform	A platform team provides infrastructure primitives so domain teams can build and serve data products independently
4. Federated Computational Governance	Global standards (data contracts, interoperability, security) enforced via automation, not central gatekeeping

13.3 Data Contracts

A data contract is a formal agreement between a data producer and consumer defining schema, quality SLAs, delivery frequency, and ownership.

- Schema: Agreed-upon field names, types, and constraints (often YAML/JSON Schema)
- SLA: Freshness guarantees, uptime, null rate thresholds
- Versioning: MAJOR.MINOR.PATCH — breaking changes increment MAJOR
- Tools: Soda Data Contracts, OpenDataMesh, custom YAML + CI validation

14. Advanced Interview & Exam Quick Reference

Critical Concepts to Master

Topic	What to Know
Inmon vs. Kimball	Top-down normalized DW → marts (Inmon) vs. Bottom-up dimensional marts → bus (Kimball)
Star vs. Snowflake	Star = fewer joins, faster; Snowflake = less redundancy, more joins
SCD Types	Type 1 = overwrite; Type 2 = version row (with eff dates + is_current); Type 6 = hybrid
Grain Declaration	Always declare fact table grain before designing. Grain = lowest level of detail per row.
ETL vs. ELT	ETL transforms before loading (on server); ELT loads raw, transforms inside DW (dbt)
CDC Methods	Log-based (most efficient), trigger-based (simple), timestamp-based (can miss deletes)
Partitioning	Range partition by date is standard for DW fact tables; enables partition pruning
Columnar Storage	Reads only needed columns; same-type data compresses well; critical

Topic	What to Know
	for OLAP performance
Conformed Dimensions	Shared dimensions (Date, Customer, Product) enabling cross-mart analysis in bus matrix
Data Vault	Hubs (keys), Links (relationships), Satellites (attributes + history); audit-friendly, parallelizable
Surrogate Keys	Integer-based system-generated keys; stable, fast, support SCD Type 2
Bridge Table	Resolves many-to-many relationships in dimensional models
Data Mesh	Domain ownership + data as product + self-serve platform + federated governance
Open Table Formats	Delta Lake (Databricks/Spark), Iceberg (multi-engine), Hudi (upserts/CDC)
Materialized Views	Pre-computed aggregates; auto query rewrite speeds up predictable workloads
Data Observability	Monitor freshness, volume, schema, distribution, lineage continuously

Data Warehouse Comprehensive Notes • Covers Foundational to Advanced Topics • For Study & Reference